

Renderide: An OpenXR-First, Data-Driven Renderer for Long-Lived Social VR Content

DOUBLESTYX

Renderide is a renderer for Resonite, a social virtual-reality sandbox whose world simulation and user content model are owned by FrooxEngine. The system replaces the Unity Built-in Render Pipeline renderer with a Rust and wgpu renderer that attaches to the host over shared-memory queues, owns windowing and XR presentation, and keeps the engine process largely unchanged. This paper describes the motivation, architecture, and rendering model of Renderide as a compatibility-oriented graphics system: it is not a general-purpose game engine, but a rendering process designed to preserve a large body of existing user-generated content while making the rendering pipeline explicitly controllable.

The main focus is an architectural separation between host simulation and renderer execution that is narrow enough to remain compatible with the existing host, yet rich enough to support a modern render graph, OpenXR-first stereo rendering, WGSL material composition, clustered forward lighting, HDR post processing, GPU-driven deformation, and headless validation. We discuss why Unity's role as a renderer became a limiting factor for Resonite: fixed shader availability, limited control over dynamic shader compilation, long-term content compatibility pressure, older XR integration constraints, and the maintenance burden of expressing a custom social-VR renderer through a general engine's built-in pipeline. We then describe Renderide's process topology, IPC contract, scene mirror, material system, render graph, asset integration model, and validation infrastructure. Finally, we identify the current limitations and future work needed before such a renderer can serve as a complete replacement for a mature Unity-based renderer.

Additional Key Words and Phrases: real-time rendering, virtual reality, OpenXR, wgpu, render graphs, content compatibility, clustered forward rendering, WGSL

1 Introduction

Real-time rendering systems are usually discussed as part of engines. The rendering subsystem is assumed to share memory, entity lifetime, asset ownership, editor tooling, and simulation timing with the rest of the runtime. Resonite uses a different boundary. It is a social VR sandbox for synchronous world building, media sharing, avatars, and collaborative user-generated content [Yellow Dog Man Studios 2023, 2026d]. FrooxEngine owns simulation and content state, while a separate renderer process communicates over an explicit inter-process contract and owns GPU submission, presentation, and input surfaces [Resonite Wiki contributors 2026a,c].

Renderide follows that model. It is a Rust renderer built on wgpu, WGSL, and OpenXR, intended as a drop-in renderer process for Resonite [DoubleStyx 2026]. It takes host-authored scene data, material updates, asset uploads, camera descriptions, and input synchronization over shared-memory queues. It then mirrors only the render-relevant subset of the world, produces view plans for desktop, head-mounted displays, and offscreen cameras, executes a graph of GPU passes, and presents the result. The host process remains authoritative for scene structure, simulation, networking, editing, and user content semantics.

This paper is framed as a systems paper rather than a performance note. Renderide is experimental, and broad numerical claims would be premature without an agreed benchmark corpus for Resonite

content. The central architectural question is what kind of renderer is needed when the primary design constraint is not a fixed game scene, but long-lived, user-generated, collaborative virtual worlds whose content must continue to render years after it was built. In that setting, compatibility constrains material semantics, shader routing, camera behavior, texture orientation, render queues, stencil state, reflection probes, video textures, skinned meshes, UI rendering, and XR presentation.

The public "Road to New Renderer" discussions for Resonite describe the larger motivation: replacing Unity as the official renderer with a system that the project controls, while preserving existing content as much as practical and enabling future features such as OpenXR support and custom shaders [Frooxius 2025a,b]. The associated candidate-evaluation process emphasizes that the renderer must be judged not only on feature checklists, but on the extent to which it can support future renderer-owned pipeline decisions [Yellow Dog Man Studios 2026b]. Renderide is one implementation of that direction: a dedicated renderer process that understands the existing host contract while allowing the graphics architecture to evolve independently from the host.

The paper makes three observations. First, a renderer for a social VR sandbox has to handle a different workload from a renderer for a conventional title: highly dynamic content, arbitrary runtime asset changes, user-authored material combinations, many cameras and render textures, and a wide range of desktop and XR output configurations. Second, the Unity Built-in Render Pipeline, while historically effective for Resonite, became a poor long-term control surface for this workload. Its shader and pipeline model exposes useful compatibility semantics, but it does not give the project complete control over runtime shader compilation, render graph ordering, XR integration, GPU resource lifetime, or cross-platform API policy [Unity Technologies 2018, 2026c,k]. Third, the IPC process split turns renderer replacement from a rewrite of the whole application into a bounded systems problem: maintain the wire contract, mirror scene data precisely, and implement a renderer that is free to use modern graphics architecture internally.

2 Background

2.1 Resonite's Renderer Boundary

Resonite separates the host from the renderer. The host submits renderable state, transforms, materials, assets, cameras, lights, and input synchronization through an IPC layer. The old renderer used Unity as the process responsible for drawing that submitted world. Resonite's Linux VR setup notes make that separation visible in practice, because the host and renderer can have different runtime constraints [Resonite Wiki contributors 2026b]. Renderide keeps the same high-level topology: a bootstrapper starts and supervises the host and renderer, the host sends small control messages through queues, bulk data moves through shared memory, and the renderer returns input and frame timing information.

This topology has two consequences. The first is positive: renderer replacement is possible without moving the whole engine. The host can continue to own simulation, network synchronization, content editing, and engine APIs. The renderer can be replaced by anything that can understand the shared protocol, map host concepts to graphics resources, and provide the same observable rendering behaviors. The second consequence is demanding: the renderer cannot assume that it owns gameplay state. It must be conservative about scene mutation, dense transform indices, asset lifetime, and the frame handshake. It is a consumer and presenter of host state, not the source of truth for the world.

The public Renderite libraries document this separation in the Unity renderer generation of the system. A shared library defines IPC and renderer-independent models, while Unity-specific code implements the rendering behavior inside Unity [Yellow Dog Man Studios 2026a,c]. The open shader repository likewise exists as reference material for future renderer work, because all existing shader behavior must be understood and reimplemented if Unity is removed [Yellow Dog Man Studios 2026e]. Renderide treats those sources as compatibility references, not as architecture to copy wholesale.

2.2 Why Unity Became a Constraint

Unity’s Built-in Render Pipeline gave Resonite a practical renderer for a long period of development. It provided a mature editor ecosystem, shader infrastructure, asset import behavior, and VR support when those capabilities were valuable. Unity’s own documentation describes BiRP as a fixed built-in pipeline with forward, deferred, and vertex-lit rendering paths whose lighting, transparency, MSAA, and shadowing behavior differ substantially by path [Unity Technologies 2026c,e]. The problem was not that Unity failed as a renderer. The problem was that Unity became the wrong abstraction boundary for a project that had already split its engine from its renderer and wanted to control the renderer’s future.

The most visible constraint is shader control. Resonite currently exposes a fixed set of shaders derived from Unity shader assets. The shader repository itself explains why arbitrary Unity custom shaders are a long-term compatibility problem: content built against arbitrary renderer-specific code cannot be promised to survive a future renderer replacement [Yellow Dog Man Studios 2026e]. Yet a new renderer is also expected to enable a better custom shader story. That implies the project needs control over shader representation, compilation, reflection, validation, and compatibility policy. A Unity BiRP renderer can load and execute Unity shaders, but it cannot serve as a neutral long-term shader platform for a renderer-independent content ecosystem.

A second constraint is render graph control. Resonite content exercises many rendering features that need careful ordering: camera stacking, render textures, UI stencil behavior, grab or scene-color effects, depth prepasses, transparent material ordering, reflection probes, video textures, offscreen captures, and XR mirror presentation. Unity exposes render queues, subshader tags, blend state, depth state, stencil state, and manual camera rendering as individual mechanisms [Unity Technologies 2015, 2026d,g,h,i,j]. Those mechanisms are useful compatibility references, but the renderer is still

expressing a custom frame through Unity’s frame. Renderide instead makes pass ordering and resource ownership first-class renderer data.

A third constraint is XR and platform control. OpenXR is the natural modern API for portable XR runtimes [Khronos OpenXR Working Group 2026]. A renderer that owns OpenXR can synchronize headset pose acquisition, input sampling, view planning, multiview rendering, mirror presentation, and failure handling around its own frame lifecycle. A renderer embedded in an older Unity project inherits Unity’s XR stack and version constraints. For Resonite, where desktop and VR modes may switch and coexist with host-visible output devices, XR is not a plugin feature; it is part of the core presentation contract.

A fourth constraint is maintenance. The public Unity renderer includes a Unity 2019-era project, dependencies for VR stacks and media playback, and renderer behavior distributed across Unity project assets, C# libraries, and ShaderLab/HLSL sources [Yellow Dog Man Studios 2026c]. That is a reasonable form for a Unity project, but it is an awkward form for an independently evolved renderer whose host already speaks a compact IPC protocol. Renderide collapses renderer-specific behavior into one Rust workspace with explicit shader sources, generated shared types, validation tests, and a renderer-owned render graph.

3 Requirements for a Replacement Renderer

The Resonite requirements discussions identify two categories of requirements: content parity and future capability [Frooxius 2025a]. Content parity includes the expected behavior of existing mesh renderers, skinned meshes, blendshapes, materials, lights, cameras, render textures, reflection probes, post-processing effects, video textures, input forwarding, stencil state, render queues, and XR output. Future capability includes OpenXR-first rendering, dynamic desktop/VR switching, custom shader support, mobile feasibility, modern graphics APIs, and deeper pipeline control.

The requirements are unusually broad because Resonite content is not a fixed set of scenes. Users build content in game, edit it collaboratively, stream assets at runtime, combine material features in ways that no single authored scene would exercise, and expect old worlds to remain usable. A renderer replacement therefore needs to operate less like a new title renderer and more like a compatibility runtime.

Table 1 summarizes the design pressures that shape Renderide. The table is not a compliance matrix. Some features remain in progress. The point is to show the categories around which the architecture is organized.

The most important requirement is not a single feature. It is control. The renderer must be able to decide where a pass appears, which resources it reads and writes, how a shader is compiled and reflected, how an XR frame is acquired, when asset integration is budgeted, and how diagnostics observe a frame. This is the axis on which a dedicated renderer differs most from a renderer expressed through another engine.

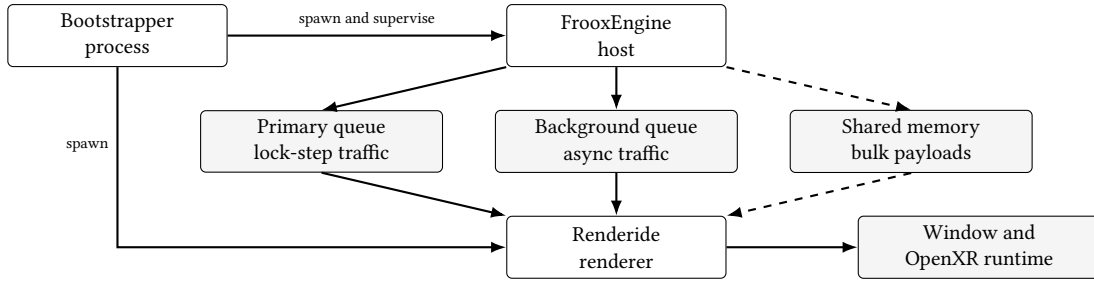


Fig. 1. Renderide preserves Resonite’s host-renderer separation. The renderer owns the GPU, window, and OpenXR runtime, while the host remains authoritative for simulation and submitted scene data.

Table 1. High-level replacement-renderer pressures that shape Renderide. The system is designed to place these concerns in explicit subsystems rather than one-off frame code.

Concern	Compatibility pressure	Renderide approach
Host integration	Preserve FrooxEngine’s authority over world state and frame cadence.	Shared-memory queues, generated wire types, lock-step frame exchange, and a render-only scene mirror.
XR presentation	Support modern VR runtimes, stereo rendering, input, haptics, and desktop mirroring.	OpenXR-owned session path, planned HMD views, multiview shader variants, and mirror blits rather than a second world render.
Materials	Preserve Unity material property names, render-state semantics, shader variants, texture defaults, and pass topology.	WGSL material roots, reflection, pass directives, explicit pipeline-state separation, and shader-specific variant-bit decoding.
Dynamic assets	Accept runtime mesh, texture, cubemap, render-texture, shader, and video updates without blocking the frame.	Budgeted asset integration, resident GPU pools, async video path, and per-frame resource preparation.
Many lights and transparent content	Support many dynamic lights while preserving MSAA and transparent material behavior.	Clustered forward lighting and explicit opaque/transparent draw planning.
Validation	Catch shader layout errors, wire-format drift, and rendering regressions outside the full application.	Build-time shader composition, source audits, cross-language generator tests, and a headless renderer suite.

3.1 Compatibility as a Rendering Problem

Compatibility in this domain is visual, temporal, and procedural. Visual compatibility means that a material or world should look close enough to its previous appearance that users recognize their content. Temporal compatibility means that the content behaves correctly as it changes over time: video textures update, procedural textures replace mips, blendshapes animate, cameras render into textures, and the host can submit different output devices across frames. Procedural compatibility means that renderer-facing APIs continue to accept the data shapes content already emits, even if the new renderer internally represents those shapes differently.

This is a stricter standard than “support a similar feature.” A renderer could support PBR, but still fail compatibility if its smoothness interpretation differs, if its reflection-probe mip selection is inconsistent, or if transparent PBS materials no longer perform their depth and cull passes in the order old content expects. A renderer could support texturing, but still fail if it interprets texture orientation or sampler wrap behavior differently from the host contract. A renderer

could support cameras, but still fail if render-texture self-sampling or camera stacking produces a different frame dependency.

Renderide therefore treats compatibility as an empirical and architectural problem. The architecture must preserve enough information to reproduce the old behavior, and the validation process must reveal cases where it does not. Some differences will be chosen improvements. Better depth precision, improved tone mapping, corrected lighting errors, or stronger resource lifetime rules may intentionally differ from the old renderer. But those differences should be explicit choices, not accidental side effects of a renderer that lacks the vocabulary to describe the old behavior.

3.2 Design Principles

Renderide’s design follows five principles.

First, renderer-owned concerns should be renderer-owned. XR frame timing, shader reflection, pipeline state, transient resource lifetimes, post-processing order, GPU profiling scopes, and presentation should not be implicit consequences of a host command. The

host requests what should be rendered; the renderer decides how to draw it.

Second, the host contract should remain bulk-oriented. Large worlds cannot be submitted as individual immediate draw commands over IPC. The renderer needs persistent mirrored state and dense draw planning. Shared memory is used for large payloads, while queues carry control flow and descriptors.

Third, hot paths should operate on compact render data. Host-shaped state is necessary for correctness at the mirror boundary, but it is not the right shape for draw execution. Renderide extracts draw packets, per-draw slabs, material batches, and frame resources that are suitable for culling, sorting, instancing, and parallel command recording.

Fourth, shader compatibility should be expressed through meta-data and validation. Material roots, pass directives, reflected bindings, defaults, texture requirements, and variant bits give the renderer structured knowledge about shader behavior. Text alone is insufficient, because the renderer must schedule snapshots, allocate bindings, choose vertex streams, and build pipeline state before a fragment runs.

Fifth, diagnostics are a core renderer subsystem. A replacement renderer will fail in content-specific ways, so it needs logging, frame timing, resource counters, shader-route visibility, asset integration summaries, and test harnesses from the beginning. Those tools turn compatibility work into an observable engineering process.

These principles are consistent with a broader shift in real-time graphics from monolithic frame code toward explicit resources, explicit synchronization, and production-oriented frame compilers. Frostbite's FrameGraph presentation framed render passes and resources as a graph to improve extensibility, alias transient memory, and keep feature code decoupled [O'Donnell 2017]; Unreal's Render Dependency Graph documentation describes a similar production convention for pass setup, resource lifetime, and execution ordering [Epic Games 2026]. Renderide's graph is smaller in scope than those engines, but it follows the same premise: a renderer that must host many optional effects and outputs should describe frame dependencies directly.

4 System Architecture

4.1 Process Model and IPC

Renderide runs as a sibling process to the host. A bootstrapper owns process lifetime and OS-level bridge services. The host and renderer communicate using a primary queue for frame-cadence traffic, a background queue for asynchronous traffic, and shared-memory regions for bulk payloads. Queue messages stay small and refer to shared-memory buffers by descriptor; large vertex buffers, index buffers, texture data, transform batches, and material batches do not travel as individual draw commands.

This design is essential for performance and for ownership. Individual draw calls cannot be proxied from the host to the renderer every frame. The renderer needs enough local state to cull, sort, batch, and prepare GPU resources. At the same time, the host remains authoritative. Renderide mirrors the render-relevant subset of the scene, not the whole engine model.

The shared wire contract is generated from host-side definitions into Rust. That generator is part of the workspace and is validated by cross-language packing tests. This is a nuanced piece of engineering, but it is central to the renderer's viability. A renderer replacement cannot tolerate silent byte-layout drift across process and language boundaries. The generator also keeps graphics dependencies out of the shared wire crate, so IPC types remain usable by host tools, tests, and future replay utilities.

4.2 Layering

Renderide's internal layering follows the process boundary. The application layer owns startup, event-loop driving, headless mode, window presentation, and process exit. The frontend owns IPC transport, session state, input conversion, output policy, lock-step gating, and decoupling decisions. The scene layer owns render spaces, transforms, cameras, lights, renderable state, material overrides, and reflection-probe requests, but no GPU resources. The backend owns the device, queues, GPU pools, material registry, render graph, frame resources, and diagnostics snapshots. The runtime ties these layers together in a fixed per-tick order.

The architectural rule is that lower renderer layers should not reach upward for live state. The scene does not submit GPU work. The backend does not own IPC queues. The frontend does not mutate GPU resources directly. Runtime orchestration captures snapshots at layer boundaries and passes them to the next phase. That discipline is particularly important because Renderide must support connected host execution, standalone execution, and headless test harness execution through the same conceptual path.

The frame order is also deliberate. The renderer first drains host commands, then integrates a budgeted slice of asset work, then maintains GPU services such as history and readback jobs, then acquires XR frame data if active, then sends begin-frame information back to the host, then plans views, collects draws, executes the graph, and presents. Figure 2 shows this flow.

4.3 Scene Mirror

The scene mirror is a compact renderer-side projection of host state. It stores render spaces, dense transform arenas, static and skinned mesh renderers, cameras, light submissions, render and material overrides, skybox state, reflection-probe tasks, and cached world matrices. It deliberately does not store GPU buffer handles or pipeline objects. That division keeps host-world semantics separate from GPU lifetime semantics.

Dense transform handling is a notable compatibility detail. The host submits transform updates by index, including removals whose order matters. The renderer cannot freely reorder these structures for convenience without breaking the meaning of later host batches. For hot draw preparation, Renderide derives compact render-state structures from the scene mirror rather than turning the host mirror itself into the draw packet format.

This choice reflects a wider principle. The renderer keeps host-shaped state at the scene boundary, then extracts render-shaped state for backend work. A direct host-object-to-draw-call mapping would be too expensive and too hard to optimize. A fully renderer-owned scene graph would duplicate authority that belongs to the

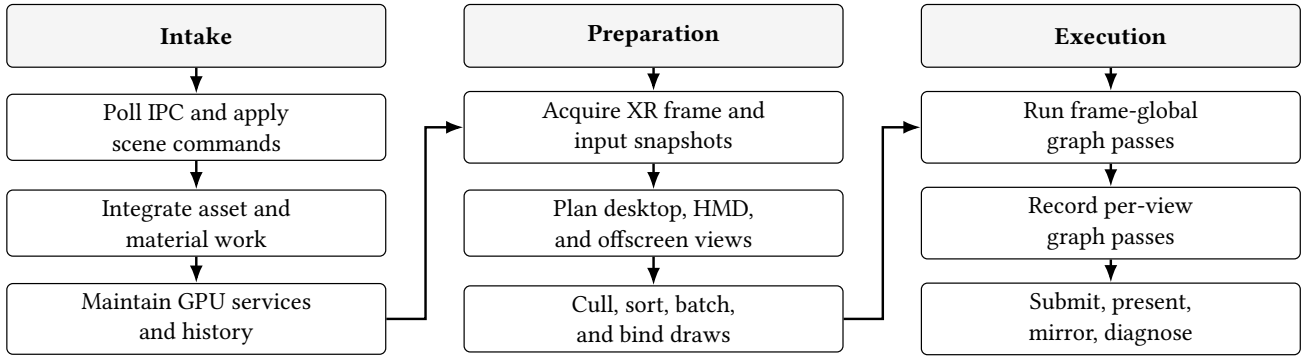


Fig. 2. Renderide’s frame lifecycle separates intake, preparation, and execution. Scene and asset state are settled before draw collection, and graph execution consumes planned views rather than live transport state.

host. Renderide uses a middle form: a faithful mirror for correctness, followed by dense draw planning for performance.

5 Rendering Architecture

5.1 wgpu and API Portability

Renderide uses wgpu as its graphics abstraction. wgpu exposes a safe Rust API over native backends and follows WebGPU’s resource and pipeline model [wgpu contributors 2026; World Wide Web Consortium 2026a]. That choice has two strategic consequences. First, the renderer can target Vulkan, Metal, Direct3D 12, and other wgpu-supported backends from a single codebase. Second, WGSL becomes the renderer’s primary shader language, aligning shader authoring with the validation and reflection model used by the graphics backend [World Wide Web Consortium 2026b]. Shader translation and composition also sit in an ecosystem of Rust tools, including Naga for shader translation and validation and naga-oil for WGSL module composition [Bevy contributors 2026; gfx-rs contributors 2026].

The choice is not free. A renderer that uses wgpu must express itself through wgpu’s available features, limits, and binding model. It cannot assume every desktop or XR feature exposed by a native API is immediately available. Renderide therefore treats device limits and feature checks as part of normal renderer startup and graph construction. Features such as variable rate shading and future mobile paths depend on backend support and must be designed as scalable capabilities rather than assumed constants.

5.2 OpenXR-First View Planning

OpenXR integration is not treated as an optional display plugin. The renderer owns session initialization, swapchain images, per-frame wait and locate calls, view geometry, controller bindings, haptics, and input translation. OpenXR’s projection-layer model explicitly couples per-view poses, fields of view, and swapchain images in the submitted frame, which makes it a natural boundary for a renderer-owned XR path [Khronos OpenXR Working Group 2026; OpenXR Tutorial contributors 2026]. When an HMD path is active, Renderide plans HMD views as first-class render targets. When multiview is available, the same draw stream can be broadcast to the two eye layers, reducing CPU submission overhead for stereo rendering; Vulkan’s multiview extension describes this family of techniques as

recording one command stream whose shader behavior varies by view index [Khronos Vulkan Working Group 2026].

The desktop mirror is intentionally a presentation operation. In a VR session, the desktop window mirrors an HMD eye rather than triggering a second world render. This preserves frame cost, avoids divergent camera state, and keeps secondary cameras from being evaluated twice. It also keeps a single conceptual source for the view that was actually submitted to the XR runtime.

Desktop, HMD, and offscreen views are all represented as planned views. The same abstraction covers render textures, camera outputs, mirrors, previews, and world-authored displays without treating each output class as a separate renderer. Each view declares its resources, executes through the graph, and is presented or copied according to its output target.

5.3 Reverse-Z and Depth Precision

Renderide uses reverse-Z depth: the near plane maps to high depth values and the far plane to low depth values. Reverse-Z is a common modern solution to the nonuniform precision of perspective depth buffers, especially when floating-point depth is available [Reed 2021]. The broader precision problem is well studied: small changes in projection math and depth conventions can materially affect the range of scenes that render without z-fighting or downstream depth artifacts [Upchurch and Desbrun 2012]. Resonite worlds can place nearby hands, UI, avatars, room-scale objects, large constructions, and distant scene elements in the same view. Depth precision is therefore a practical compatibility issue rather than a purely numerical preference.

Reverse-Z affects projection matrices, clear values, depth comparisons, Hi-Z pyramid construction, and shader helper conventions. The renderer treats it as a global convention. That reduces the chance that an individual pass silently uses forward-Z assumptions, which would be particularly damaging for occlusion, depth-aware post effects, and material features that sample scene depth.

5.4 Render Graph

The render graph is Renderide’s per-frame compiler. Passes declare their resources and access modes; the graph schedules them, allocates transient resources, manages persistent history, records

command encoders, drains uploads, and submits work. This is the mechanism that replaces a collection of ad hoc render callbacks with explicit data dependencies, following the same general production argument made by frame-graph systems: resource lifetime, barriers, and optional pass insertion become graph properties rather than scattered imperative state [Epic Games 2026; O'Donnell 2017].

Renderide distinguishes frame-global passes from per-view passes. Frame-global work includes tasks such as mesh deformation, light preparation, and history maintenance that should run once per tick. Per-view work includes world rendering, depth and color snapshots, post processing, and composition for each planned view. The graph cache keys its compiled variants on durable frame-shape inputs such as surface extent, MSAA sample count, multiview mode, and target formats. When those inputs change, a new graph variant is compiled and cached.

The graph is especially important for future compatibility. Some features need explicit resource ordering: scene-color sampling, refraction, motion data, reflection captures, camera stacking, and mirror or portal rendering. Without a graph, those features tend to become one-off paths that accidentally duplicate passes or depend on global mutable state. With a graph, they become resource declarations and scheduled passes.

5.5 Clustered Forward Lighting

Renderide uses clustered forward lighting, following the broad family of techniques introduced by clustered deferred and forward shading [Olsson et al. 2012]. Lights are binned into a three-dimensional grid in view space. During forward shading, a fragment locates its cluster and iterates only the lights assigned to that cluster. This gives the renderer a many-light path without inheriting the semi-transparent-object and hardware-MSAA limitations Unity documents for its deferred BiRP path [Unity Technologies 2026e]. Deferred approaches have a long lineage, from hardware ideas that shaded after visibility resolution to G-buffer postprocess structures [Deering et al. 1988; Saito and Takahashi 1990]; Renderide's choice is not a rejection of that lineage, but a compatibility decision for MSAA, transparent materials, and shader diversity.

This design is well suited to Resonite's requirements. A deferred renderer would make many dynamic lights cheap for opaque surfaces, but it would complicate transparent materials, MSAA, stylized shading, and the varied material set needed for existing content. A naive multi-pass forward renderer would preserve material flexibility but scale poorly with many lights. Clustered forward rendering keeps the material model forward while bounding the per-fragment light set.

The renderer's current lighting path supports the core dynamic-light categories needed for compatibility work. Shadowing, richer light cookies, custom falloff controls, and more advanced light types remain future work. The important architectural choice is that light preparation is a renderer-owned pass, not a side effect of host draw submission.

5.6 Visibility, Sorting, and Batching

The world-mesh planner combines frustum culling, temporal Hi-Z occlusion information, material resolution, render-queue classification, sorting, batching, and instancing decisions. This subsystem is pure CPU planning. It consumes scene state and previous-frame visibility data, then produces sorted draw packets for the graph's forward pass.

The use of Hi-Z follows a common temporal pattern: build a hierarchical depth representation from a resolved depth target, then use it to conservatively reject occluded objects in a later frame. Hierarchical Z-buffer visibility was introduced as an image-space pyramid paired with object-space hierarchy to exploit coherence in visibility computation [Greene et al. 1993]. The detail that matters in Renderide is consistency with reverse-Z. The pyramid operation must summarize depth in the direction that preserves conservative rejection under the renderer's depth convention.

Sorting is not a single comparator. Opaque work benefits from batching and state locality, with front-to-back ordering useful when it reduces overdraw. Transparent work must preserve blending correctness and material-specific pass topology. UI and stencil-heavy content may impose ordering constraints that are more important than conventional mesh sorting. Renderide therefore treats draw arrangement as a renderer subsystem rather than a direct reflection of host entity order.

6 Materials and Shader Compatibility

6.1 WGSL Material Sources

Renderide's shader sources are written in WGSL and composed with reusable modules. Material roots correspond to host shader assets, while shared modules provide frame data, draw data, mesh transforms, PBS lighting, UI and text helpers, skybox logic, post-processing support, and sampling utilities. WGSL is validated and reflected during the renderer build, producing embedded metadata that the runtime uses for bindings, vertex stream requirements, material uniform packing, and pass construction. This mirrors the direction of modern graphics APIs, where shader modules, bind groups, pipeline layouts, and entry-point reflection are explicit compile-time and runtime objects rather than hidden driver-side conventions [wgpu contributors 2026; World Wide Web Consortium 2026a,b].

The material library is large because compatibility is large. It includes PBS families, unlit shaders, overlay and UI shaders, text shaders, skybox materials, toon shading, wireframe materials, filters, volume and refraction effects, fur-family shaders, and many variants that exist because old content can refer to them. A clean renderer for a new application might replace many of these with a smaller material system. A replacement renderer for existing Resonite content cannot begin there. It must first preserve the semantic surface users already rely on.

6.2 Shader Routing and Variants

Shader identity comes from host assets. Renderide routes those identities to embedded WGSL material stems and decodes shader-specific variant bits. This distinction is important: the asset route chooses the material family, while variant metadata selects branches within that shader's own keyword vocabulary. Treating variant bits

as a global enum would be wrong, because Unity shader keywords are shader-local and sorted according to shader metadata rules.

The renderer reserves a material uniform for the raw variant bitmask where needed, and WGSL helper modules decode it into shader-local feature predicates. This keeps keyword behavior inside the material and keeps pipeline state outside it. It also gives the renderer a clear path toward future shader APIs: a custom shader system needs explicit metadata, reflection, validation, and variant representation, not just arbitrary text passed to a driver.

6.3 Pass Directives and Render State

Unity ShaderLab expresses pipeline behavior through subshaders, passes, tags, blend state, depth state, stencil state, culling, color masks, and related directives [Unity Technologies 2018, 2026g,h,i,j]. Renderide preserves this kind of structure through material pass directives in WGSL comments. Each directive describes a pass type, entry point, and render-state policy. The build step parses these directives and the runtime turns them into concrete pipelines.

This is more than a convenience. Some materials are multi-pass by nature. Transparent PBS families may require depth prepasses, different cull modes, or ordered front/back draws. UI content depends heavily on stencil operations. Some effects need scene color or scene depth snapshots. If the renderer treated every material as a single implicit forward pass, it would fail compatibility for the content that depends on authored pass topology.

Renderide also separates pipeline state from shader uniforms. Properties such as blend factors, culling, depth writes, stencil operations, color masks, and polygon offsets affect pipeline construction. Properties such as colors, cutoffs, smoothness, UV transforms, texture bindings, and shader feature flags affect material data. The build system rejects material uniform layouts that accidentally include pipeline-state fields. This separation keeps pipeline cache keys meaningful and avoids writing uniform data that shaders cannot consume.

6.4 Physically Based and Stylized Shading

Renderide's PBS implementation draws on the same family of real-time physically based shading used in contemporary engines. The theoretical frame begins with the rendering equation and early physically motivated reflectance models [Cook and Torrance 1982; Kajiya 1986]. Practical real-time material systems then rely on economical approximations: Schlick's Fresnel approximation and related BRDF simplifications [Schlick 1994], microfacet reflection and transmission models [Walter et al. 2007], and masking-shadowing terms such as those analyzed by Heitz [2014]. Disney's shading work influenced the language of artist-facing material parameters [Burley 2012]; Unreal Engine 4 and Frostbite course notes popularized practical split-sum image-based lighting and production PBR workflows for games [Karis 2013; Lagarde and de Rousiers 2014]. Filament's public material documentation is another useful example of a modern engine reducing the same theory into compact real-time material inputs [Google Filament contributors 2026].

However, Renderide's material problem is not simply "implement modern PBR." Resonite content includes stylized toon materials, matcaps, unlit overlays, UI, text, volume effects, wireframe styles,

filters, and shaders whose behavior is shaped by Unity-era compatibility. For this reason the renderer is organized around material families and shared modules. The PBS family can share BRDF, normal, displacement, cluster-light, and IBL code. Toon materials can share their own lighting and outline modules. UI and text can share clipping and signed-distance-field behavior, a technique lineage that includes Valve's distance-field vector-texture work [Green 2007]. Compatibility and maintainability both depend on reducing duplicate shader logic while preserving shader-specific semantics.

6.5 Scene Color, Depth, and Material Side Effects

Many material systems can be described as a pure function of material parameters and geometry. Resonite compatibility requires a broader view. Some material families need scene color, scene depth, grab-like inputs, reflection information, stencil state, or special interactions with UI and text rendering. Transparent and composited effects also inherit decades-old alpha-compositing constraints: blend state participates in the algebra by which separately rendered images are combined [Porter and Duff 1984; Unity Technologies 2026h]. These are requests for renderer resources at specific points in the frame, not ordinary material uniforms.

Renderide's material metadata records these needs so the graph can schedule the required resources. A refraction material that samples scene color implies that a color snapshot must exist before the material reads it. A material that clips or intersects against scene depth implies that a depth snapshot must be available in the correct convention. A UI material that writes or tests stencil state implies that the pass cannot be merged with a generic forward draw if doing so changes stencil ordering. This is why compatibility material work is the intersection of shader reflection and render-graph planning.

In Unity, many of these behaviors were distributed across ShaderLab pass structure and pipeline conventions. Renderide reconstructs the relevant semantics in a renderer-owned form. The compatibility target is the observable contract that user content depends on, not Unity's internal implementation.

6.6 Texture Semantics and Defaults

Texture behavior is another compatibility surface. Materials refer to texture properties by name. Missing textures have expected defaults. Texture coordinate transforms carry Unity-style scale and offset fields. Sampler wrap behavior, host mip bias, cubemap orientation, volume texture orientation, tangent-space normal-map conventions, and render-texture sampling conventions can all affect visible content. In particular, normal-mapped materials depend on a consistent tangent basis; MikkTSpace has become a common reference implementation for matching baked normal maps to runtime shading [Mikkelsen 2026].

Renderide therefore treats texture bindings as reflected material data rather than informal shader code. Texture and sampler pairs occupy predictable binding slots. Defaults are declared for named texture properties. Host mip bias is represented in material data and used by sampling helpers. Cubemap storage orientation is handled separately from ordinary 2D material textures. In aggregate, these details decide whether old materials survive the renderer migration.

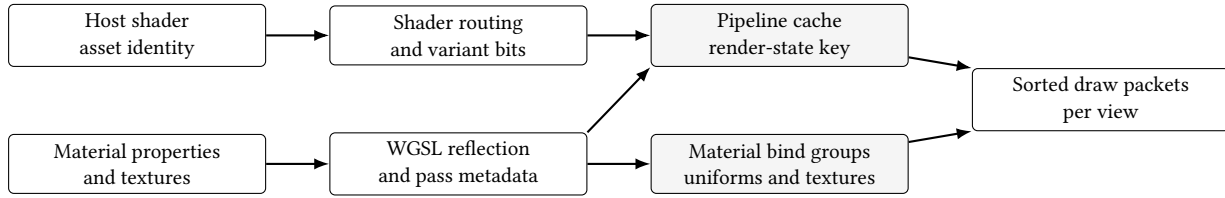


Fig. 3. Material resolution separates shader routing, WGS reflection, pipeline state, and material bind groups. This separation is central to preserving Unity-style material semantics without embedding pipeline state in shader uniforms.

7 Assets, Dynamic Content, and GPU Resources

7.1 Runtime Asset Integration

Resonite content is dynamic. Meshes, textures, cubemaps, render textures, video textures, material properties, and shader routes can arrive while the renderer is running. Assets may be uploaded progressively, updated procedurally, or removed. The renderer must integrate them without blocking presentation for unbounded time.

Renderide uses a cooperative asset-integration phase. The host submits asset work through IPC and shared memory. The renderer queues the work, drains a budgeted slice each tick, uploads GPU resources, updates resident pools, and makes the results visible to later draw collection. The budget is configurable and can shrink when the renderer is decoupled from the host, so that a burst of asset work does not prevent the user from receiving frames.

This model is different from synchronous asset loading in a conventional game. The renderer cannot assume a loading screen. It must remain responsive while host content changes. The asset pipeline therefore has to handle partially available data, repeated uploads, changing texture mips, render-texture allocation, and material updates as normal events.

7.2 Meshes, Skinning, and Blendshapes

Renderide supports static meshes and skinned meshes with blendshape data. Skinning and blendshape work runs as GPU compute before per-view rendering. The output feeds forward drawing through deformed buffers and per-draw uniforms. Computing deformation once per frame avoids repeated stereo and secondary-camera work when the same deformed mesh is visible in multiple views. The current approach is closest to conventional linear blend skinning plus blendshape accumulation, while planned dual-quaternion support would address well-known volume-loss artifacts in matrix blending [Kavan et al. 2008]. Blendshape support is likewise a compatibility requirement rather than a specialty facial-animation feature; the literature treats blendshapes as a simple but pervasive linear model for expressive deformation [Lewis et al. 2014].

The renderer also needs to respect host mesh conventions: positions, normals, tangents, colors, multiple UV channels, submeshes, bone weights, and blendshape frames. These are compatibility details. Missing a vertex stream may not break a test scene, but it can break old user content that depends on a specific material reading a specific channel.

Future work includes broader bone-weight support and dual-quaternion skinning. The current architecture leaves room for those

changes because deformation is already an explicit frame-global service rather than embedded in individual material shaders.

7.3 Texture and Video Data

Textures include 2D images, cubemaps, 3D textures, render textures, and video textures. Renderide’s texture path handles host byte formats, mip levels, subresource uploads, compressed formats supported by the adapter, sampler state, and texture defaults used by materials. Block-compressed texture support is a platform-facing compatibility issue because BC formats, ASTC, and container formats such as KTX reduce bandwidth and memory while requiring exact format and block-size handling [Khronos Group 2012, 2026; Microsoft 2022]. A material sample must also respect renderer conventions such as texture orientation and host-supplied mip bias.

Video textures are particularly important for social VR content, where worlds often include shared media. Renderide provides video-texture support behind an opt-in feature using a GStreamer-backed decoding path. GStreamer is structured around media pipelines and application-facing sinks, which makes it a practical fit for a renderer that needs decoded video frames and raw audio data rather than direct output to a media player’s window or audio device [GStreamer contributors 2026a,b]. When that feature is disabled, the renderer still accepts video texture commands and creates placeholders, preserving the wire behavior without requiring every build to link platform media libraries. The old Unity renderer notes that video playback relied in part on external media components, including a commercial component that could not be redistributed in full [Yellow Dog Man Studios 2026c]. Renderide’s design makes this a renderer-owned optional subsystem with explicit build-time dependencies.

7.4 GPU Pools and Resource Budgets

Per-frame allocation is hostile to predictable VR rendering. Renderide uses resident GPU pools for meshes, textures, cubemaps, render textures, and video textures, plus frame-resource managers for transient per-frame data. The goal is to move allocation to initialization, asset integration, or explicitly budgeted phases, leaving the draw and record path to reuse existing resources.

Resource pooling also gives the renderer a place to apply VRAM budgets and eviction policy. A renderer for user-generated worlds cannot assume that content authors have optimized every asset for a fixed scene. The system has to defend itself against churn and memory pressure while preserving as much content as it can.

7.5 Offscreen Cameras and Render Textures

Render textures are not a secondary feature in Resonite. They are used for world cameras, interactive displays, mirrors, avatar previews, UI surfaces, and content-authored visual effects. Unity’s camera and render-texture APIs make this pattern explicit: cameras can render to texture targets, render textures can be used as GPU resources, and manual camera rendering exists for precise ordering use cases [Unity Technologies 2015, 2022, 2026f]. The renderer must support additional views that render into texture resources, camera stacking by depth, viewport rectangles, clear modes, and eventual readback or host-visible use.

Renderide models these outputs as planned views rather than as a separate mini-renderer. The same graph that draws the desktop or HMD can draw an offscreen view with a different target, clear policy, viewport, and post-processing choice. This keeps render-texture cameras on the same material, scene-snapshot, lighting, and post-processing paths as primary views. It also keeps offscreen work visible to batching, profiling, and graph analysis.

Camera history is a subtle compatibility problem. Some content expects a camera that samples its own render texture to see the previous frame rather than the image currently being written. A renderer can avoid same-pass self-sampling, but full self-history semantics require explicit double-buffering policy. Renderide’s planned-view model gives that policy a natural place to live: a camera output can resolve to the appropriate current target and history target before graph execution begins.

7.6 Input and Output as Renderer Responsibilities

Because the renderer owns the window and XR session, it also observes keyboard, mouse, IME, controller, and headset input before the host does. The renderer’s job is not to interpret that input as gameplay. It translates platform and OpenXR state into the shared input structures expected by the host, then sends those structures as part of the frame handshake.

This creates an important timing requirement. XR poses used for rendering and XR poses sent to the host should refer to the same frame snapshot when possible. If the renderer samples the headset for rendering and later samples different input for the host, the simulation and display can disagree. Renderide addresses this by acquiring OpenXR frame data before the lock-step begin-frame exchange, so input and planned views are grounded in the same XR frame.

8 Image Quality and Post Processing

8.1 HDR Scene Color and Tone Mapping

Renderide renders world color into an HDR scene-color target before display mapping. Post-processing effects operate in that HDR domain and the final composition path maps the result into the surface format. This structure allows bloom and exposure to operate on values above display white, and it keeps the display blit responsible for final surface conversion.

The renderer currently supports ACES-style tone mapping and AgX-style mapping. ACES is a well-established color-management system in film and graphics workflows [Academy Color Encoding System 2026]; AgX entered mainstream open-source DCC practice

through Blender’s color-management update and is documented as a view transform for mapping scene-referred values into display-referred output [Blender Foundation 2023, 2026]. Tone-mapping itself has a long graphics history, with photographic operators such as Reinhard’s motivating the display of HDR radiance on low-dynamic-range media [Reinhard et al. 2002]. In Renderide, tone-mapping modes are renderer settings rather than hard-coded display behavior, preserving room for content compatibility and future HDR output paths.

8.2 Bloom, Auto-Exposure, and Motion Blur

Bloom is implemented as an HDR downsample and upsample chain. It extracts bright content before tone mapping, spreads it across a pyramid, and composites it back into the scene color. Unity’s post-processing documentation describes bloom in the same perceptual role, and Jimenez’s SIGGRAPH course notes on Call of Duty: Advanced Warfare are a widely cited production reference for downsample/upsample bloom and high-quality post pipelines [Jimenez 2014; Unity Technologies 2026a]. Auto-exposure computes a luminance distribution and adapts exposure over time so the tone mapper receives a more stable signal. Motion blur is a screen-space post effect, with desktop enabled by default and VR use treated more cautiously because headset motion and comfort constraints differ from monitor presentation. The screen-space velocity-buffer model follows the family of real-time reconstruction filters described by McGuire et al. [2012].

These effects are graph nodes rather than hidden post hooks. That means their inputs and outputs are explicit: scene color, depth snapshots, history, settings, and intermediate textures all participate in graph scheduling. The same structure supports enabling, disabling, or reordering effects by changing the post-process chain rather than rewriting the frame.

8.3 Ambient Occlusion and Indirect Light

Renderide includes Ground Truth Ambient Occlusion, a screen-space technique for estimating local indirect occlusion. The method follows the real-time rendering lineage described by Jimenez and collaborators [Jimenez et al. 2016]: reconstruct view-space depth and normals, sample occlusion over screen-space directions, denoise, and apply the result before tone mapping. In a dynamic user-generated world, screen-space AO is a pragmatic complement to baked or probe-based lighting. It provides contact darkening without requiring content authors to bake every environment.

For lower-frequency indirect light, Renderide uses image-based lighting and spherical harmonics. Reflection probes and skyboxes provide cubemap inputs for specular filtering and SH2 diffuse ambient. Image-based lighting entered production graphics through work such as Debevec’s HDR environment capture and compositing pipeline [Debevec 1998]; low-frequency irradiance maps can be represented compactly using spherical harmonics, as shown by Ramamoorthi and Hanrahan [Ramamoorthi and Hanrahan 2001, 2002]. Spherical harmonic lighting is a compact representation of low-frequency illumination, widely used in real-time graphics since the early precomputed radiance transfer literature and popularized in practical notes such as Sloan’s [Sloan 2008]. Renderide uses GPU

jobs to project environment sources into coefficients and returns results through the host-renderer protocol when necessary.

8.4 Reflection Probes and Skyboxes

Reflection probes are a compatibility-critical feature for PBS materials. They provide prefiltered specular lighting across roughness levels and ambient lighting through SH coefficients. Renderide supports baked and rendered probe sources, skybox-driven IBL, and cubemap filtering. This combination follows the production PBR pattern of separating diffuse irradiance, prefiltered specular environment radiance, and BRDF terms into data that can be sampled efficiently at runtime [Google Filament contributors 2026; Karis 2013; Lagarde and de Rousiers 2014]. The skybox system handles several families, including solid color, gradients, procedural atmospheric forms, projection 360 content, and cubemap/equirectangular sources.

Probe and skybox work feeds material lighting, reflection captures, SH projection, and offscreen tasks in addition to visible background rendering. Treating those operations as graph and GPU-job services makes them available to the renderer without coupling them to a single camera pass.

9 Diagnostics and Validation

9.1 Logging and HUD Diagnostics

Renderide exposes diagnostic state through file-first logs under component-specific directories and a Dear ImGui diagnostics overlay. The overlay includes frame timing, renderer stats, shader routes, draw state, GPU memory information, scene transforms, texture inspection, watchdog status, and XR failure counters. Diagnostics are captured through snapshots at layer boundaries rather than by borrowing arbitrary live state from the frame.

Renderer bugs in a VR application are often timing-sensitive, adapter-specific, or content-specific. The diagnostics overlay and logs make frame behavior observable without requiring a debugger attached to every report.

9.2 Profiling

Renderide includes optional Tracy integration for CPU spans and GPU timestamp scopes. CPU instrumentation records major frame phases and hot paths, while GPU scopes measure graph execution phases where adapter support permits. The GPU path builds on wgpu-profiler's timer-query scope model and can report results into Tracy when the feature is enabled [Sowa 2026; wgpu-profiler contributors 2026]. When disabled, the instrumentation compiles away or has near-zero cost.

The profiling model reflects a general performance principle: optimize by measurement first. The renderer architecture creates clear measurement regions - IPC, asset integration, view planning, draw collection, graph execution, post processing, presentation - so that performance work can distinguish CPU submission cost, GPU shader cost, bandwidth, resource churn, overdraw, synchronization, and frame pacing. This is consistent with common graphics-performance guidance: a frame-rate problem must first be localized to CPU work, GPU work, memory bandwidth, synchronization,

shader complexity, overdraw, or present pacing before a fix is chosen [Unity Technologies 2026b].

9.3 Build-Time and Headless Validation

Renderide validates shader sources during the build. WGSL is composed, reflected, and checked for binding conventions and material-state errors. Additional source audits cover shader-module hygiene, material UV handling, clustered-light shader layout, and related invariants. This is necessary because the material library is large and compatibility bugs often arise from small inconsistencies across shader families.

The renderer also includes a headless test harness that acts as a minimal host. It exercises the real IPC protocol, submits scenes, starts the renderer, captures output, and compares against golden references. This is a significant architectural advantage of the process split. The renderer can be tested end-to-end without launching the full application, but still through the same queues and shared-memory contract that the real host uses.

Continuous integration builds and tests the Rust workspace and the C# generator across major desktop platforms. The generator tests are particularly important because they protect the shared wire format. A renderer can recover from many visual bugs, but it cannot recover from the host and renderer disagreeing on message layout.

10 Discussion

10.1 What Renderide Accomplishes

Renderide's main accomplishment is not a single rendering feature. It demonstrates that the Resonite renderer boundary can host a modern renderer that is independent from Unity while still respecting the existing host. The system owns the GPU device, OpenXR session, material compilation path, render graph, asset pools, post-processing chain, and diagnostics. It does so while preserving the host's authority over simulation and content.

The architecture also creates a realistic path for future custom shader work. A custom shader system for a long-lived social platform cannot be a raw driver-language escape hatch. It needs validation, reflection, capability negotiation, metadata, compatibility policy, and renderer-independent representation choices. Renderide does not solve that whole problem, but it builds the renderer-side machinery that such a system would need: shader composition, reflected bindings, pass metadata, pipeline-state separation, variant handling, and graph-aware resource requirements.

Finally, Renderide gives Resonite a renderer where XR is not inherited from a larger engine's release cadence. OpenXR frame acquisition, stereo view planning, multiview decisions, controller input, haptics, and mirror presentation live in the renderer's own frame lifecycle. That is a more appropriate basis for a social VR platform that expects desktop, VR, and offscreen outputs to coexist.

10.2 Why the Renderer Is Not a Game Engine

Renderide has no editor, no world simulation, no asset authoring model, no network ownership, and no general entity-component system. Its scene model is a renderer mirror. Its job is to render the host's submitted state with high fidelity and predictable timing.

This narrowness is a strength. A general engine would have to map FrooxEngine’s world into another world model, creating two authorities for content. Renderide instead maps host render data into GPU-ready state. It can optimize and cache aggressively on the renderer side, but it does not reinterpret the simulation. The result is a smaller boundary and a clearer responsibility split.

10.3 Risks

The largest risk is compatibility breadth. Existing Resonite content covers years of accumulated material behavior, shader quirks, render-order expectations, texture conventions, camera interactions, and user workflows. Passing a curated test scene is not enough. A replacement renderer needs broad content exposure and a way to classify visual differences as acceptable improvements, known limitations, or compatibility regressions.

A second risk is feature incompleteness. Shadows, full LOD behavior, mirror and portal polish, foveated rendering, HDR display output, richer light cookies, mobile support, expanded input devices, and the final custom shader architecture remain active or future work. Because Renderide is explicit about these systems, they are tractable additions, but they still require implementation and validation.

A third risk is dependency and platform variance. `wgpu` provides portability, but backend behavior, adapter limits, shader compilation behavior, and XR runtime interactions still vary across platforms. The renderer’s CI and headless suite reduce that risk, but broad hardware coverage remains essential for a renderer intended to replace a mature Unity deployment.

11 Limitations and Future Work

Shadows and light cookies. The renderer already represents light types and shadow metadata. Full real-time shadow-map rendering, soft shadows, point and spot cookies, RGB cookies, and shadow reuse across cameras remain future work. Shadow mapping originates with Williams’s depth-map formulation, with percentage-closer filtering and cascaded/parallel-split variants addressing aliasing and large view ranges in later real-time systems [Reeves et al. 1987; Williams 1978; Zhang et al. 2007]. The render graph should treat shadow maps as first-class resources so that a light’s shadow can be rendered once per frame and reused by multiple views where valid.

LOD and visibility hierarchy. Level-of-detail selection is required for compatibility and scalability. A future system should select renderers or mesh variants by projected size, support transition policies where appropriate, and integrate with culling so that LOD decisions do not duplicate visibility work.

Mirrors and portals. Offscreen cameras and render textures provide much of the substrate for mirror-like content, but dedicated mirror and portal behavior requires careful treatment of view transforms, clipping, recursion policy, VR efficiency, stencil or scissor alternatives, and scene-color interactions. Screen-space ray tracing and stochastic screen-space reflections provide useful comparisons for reflection-like effects, but their limitations also illustrate why

explicit secondary views remain necessary for true mirrors and portals [McGuire and Mara 2014; Stachowiak and Uludag 2015].

Foveated rendering and mobile. OpenXR gives the renderer a portable XR foundation, but foveated rendering depends on backend support such as variable rate shading, quad-view presentation, or multi-view alternatives. Gaze-tracked and fixed foveated rendering have been studied as a way to reduce VR shading cost while preserving perceptual quality [Arm 2026; Patney et al. 2016]. Mobile and standalone headset support will also require attention to process topology, memory budgets, shader variants, thermal constraints, and API support.

Custom shaders. The long-term custom shader design is larger than a renderer feature. It must decide what representation the host stores, what compatibility guarantees exist, what capabilities are portable, how compilation is cached, how security and validation work, and how alternative renderers participate. Renderide’s WGSL material system is a practical renderer foundation, but not the final content-facing language policy.

Content-scale validation. The headless golden suite is necessary but not sufficient. A replacement renderer ultimately needs a larger compatibility corpus drawn from real Resonite content, with tools for classifying differences and tracking regressions. Some differences will be intentional, especially where the new renderer improves correctness or precision; others will need compatibility modes.

12 Conclusion

Renderide is best understood as a renderer replacement architecture for a long-lived social VR platform. Its value lies in the boundary it preserves and the control it gains. FrooxEngine remains the owner of simulation and content. The renderer becomes a dedicated Rust, `wgpu`, WGSL, and OpenXR process with explicit resource ownership, material reflection, render graph scheduling, budgeted asset integration, stereo view planning, diagnostics, and headless validation.

The old Unity BiRP renderer gave Resonite a working rendering path, but it could not be the final abstraction for a project that needs control over shaders, XR, platform policy, render ordering, and long-term compatibility. Renderide addresses that constraint by turning renderer behavior into project-owned systems rather than Unity-mediated behavior. Compatibility remains difficult, but the renderer now has explicit contracts, observable frame structure, and a graphics architecture that can evolve while preserving existing content.

References

- Academy Color Encoding System. 2026. ACES System Overview. <https://docs.acescentral.com/background/overview/>
- Arm. 2026. Foveated Rendering. <https://developer.arm.com/community/arm-community-blogs/b/mobile-graphics-and-gaming-blog/posts/white-paper-foveated-rendering>
- Bevy contributors. 2026. naga-oil. https://github.com/bevyengine/naga_oil
- Blender Foundation. 2023. Blender 4.0: Color Management. https://developer.blender.org/docs/release_notes/4.0/color_management/
- Blender Foundation. 2026. Displays and Views. https://docs.blender.org/manual/en/dev/render/color_management/displays_views.html

- Brent Burley. 2012. Physically-Based Shading at Disney. SIGGRAPH Courses: Practical Physically Based Shading in Film and Game Production. <https://disneyanimation.com/publications/physically-based-shading-at-disney/>
- Robert L. Cook and Kenneth E. Torrance. 1982. A Reflectance Model for Computer Graphics. *ACM Transactions on Graphics* 1, 1 (1982), 7–24. doi:10.1145/357290.357293
- Paul E. Debevec. 1998. Rendering Synthetic Objects into Real Scenes: Bridging Traditional and Image-Based Graphics with Global Illumination and High Dynamic Range Photography. In *Proceedings of SIGGRAPH '98*. ACM, New York, NY, USA, 189–198. <https://www.pauldebevec.com/Research/IBL/>
- Michael Deering, Stephanie Winner, Bic Schediwy, Chris Duffy, and Neil Hunt. 1988. The Triangle Processor and Normal Vector Shader: A VLSI System for High Performance Graphics. In *Proceedings of the 15th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '88)*. ACM, New York, NY, USA, 21–30. doi:10.1145/54852.378468
- DoubleStyx. 2026. Renderide. <https://github.com/DoubleStyx/Renderide>
- Epic Games. 2026. Render Dependency Graph in Unreal Engine. <https://dev.epicgames.com/documentation/en-us/unreal-engine/render-dependency-graph-in-unreal-engine>
- Frooxius. 2025a. Road to New Renderer: Phase 1 Requirements. <https://github.com/Yellow-Dog-Man/Resonite-Issues/discussions/5831>
- Frooxius. 2025b. The Road to a New Renderer: Help Us Get Away from Unity for Good. <https://github.com/Yellow-Dog-Man/Resonite-Issues/discussions/5830>
- gfx-rs contributors. 2026. Naga: Universal Shader Translation in Rust. <https://github.com/gfx-rs/naga>
- Google Filament contributors. 2026. Filament Materials. <https://google.github.io/filament/main/materials.html>
- Chris Green. 2007. Improved Alpha-Tested Magnification for Vector Textures and Special Effects. SIGGRAPH Courses: Advanced Real-Time Rendering in 3D Graphics and Games. https://steamcdn-a.akamaihd.net/apps/valve/2007/SIGGRAPH2007_AlphaTestedMagnification.pdf
- Ned Greene, Michael Kass, and Gavin S. P. Miller. 1993. Hierarchical Z-Buffer Visibility. In *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '93)*. ACM, New York, NY, USA, 231–238. doi:10.1145/166117.166147
- GStreamer contributors. 2026a. Application Development Manual. <https://gstreamer.freedesktop.org/documentation/application-development/>
- GStreamer contributors. 2026b. appsink. <https://gstreamer.freedesktop.org/documentation/app/appsink.html>
- Eric Heitz. 2014. Understanding the Masking-Shading Function in Microfacet-Based BRDFs. *Journal of Computer Graphics Techniques* 3, 2 (2014), 48–107. <https://jcgt.org/published/0003/02/03/>
- Jorge Jimenez. 2014. Next Generation Post Processing in Call of Duty: Advanced Warfare. SIGGRAPH Courses: Advances in Real-Time Rendering in Games. <https://www.advances.realtimerendering.com/s2014/index.html>
- Jorge Jimenez, Xian-Chun Wu, Angelo Pesce, and Adrian Jarabo. 2016. Practical Real-Time Strategies for Accurate Indirect Occlusion. SIGGRAPH Courses: Advances in Real-Time Rendering. <https://www.activision.com/cdn/research/PracticalRealtimeStrategiesTRfinal.pdf>
- James T. Kajiya. 1986. The Rendering Equation. In *Proceedings of the 13th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '86)*. ACM, New York, NY, USA, 143–150. doi:10.1145/15922.15902
- Brian Karis. 2013. Real Shading in Unreal Engine 4. SIGGRAPH Courses: Physically Based Shading in Theory and Practice. <https://blog.selfshadow.com/publications/s2013-shading-course/>
- Ladislav Kavan, Steven Collins, Jiri Zara, and Carol O'Sullivan. 2008. Geometric Skinning with Approximate Dual Quaternion Blending. *ACM Transactions on Graphics* 27, 4, Article 105 (2008), 23 pages. doi:10.1145/1409625.1409627
- Khronos Group. 2012. Khronos Releases ASTC Next-Generation Texture Compression Specification. <https://www.khronos.org/news/press/khronos-releases-atcsc-next-generation-texture-compression-specification>
- Khronos Group. 2026. KTX File Format Specification. <https://registry.khronos.org/KTX/specs/2.0/ktxspec.v2.html>
- Khronos OpenXR Working Group. 2026. The OpenXR 1.1 Specification. <https://registry.khronos.org/OpenXR/specs/1.1-khr/html/xrspec.html>
- Khronos Vulkan Working Group. 2026. VK_KHR_multiview. https://registry.khronos.org/vulkan/specs/latest/man/html/VK_KHR_multiview.html
- Sebastien Lagarde and Charles de Rousiers. 2014. Moving Frostbite to Physically Based Rendering 3.0. SIGGRAPH Courses: Physically Based Shading in Theory and Practice. <https://www.ea.com/frostbite/news/moving-frostbite-to-pb>
- J. P. Lewis, Ken Anjyo, Taehyun Rhee, Mengjie Zhang, Fred Pighin, and Zhigang Deng. 2014. Practice and Theory of Blendshape Facial Models. In *Eurographics 2014 - State of the Art Reports*. The Eurographics Association, Goslar, DEU, 199–218. doi:10.2312/egst.20141042
- Morgan McGuire, Padraic Hennessy, Michael Bukowski, and Brian Osman. 2012. A Reconstruction Filter for Plausible Motion Blur. In *Proceedings of the ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games (I3D '12)*. ACM, New York, NY, USA, 135–142. doi:10.1145/2159616.2159639
- Morgan McGuire and Michael Mara. 2014. Efficient GPU Screen-Space Ray Tracing. *Journal of Computer Graphics Techniques* 3, 4 (2014), 73–85. <https://jcgt.org/published/0003/04/04/>
- Microsoft. 2022. Block Compression (Direct3D 10). <https://learn.microsoft.com/en-us/windows/win32/direct3d10/d3d10-graphics-programming-guide-resources-block-compression>
- Morten S. Mikkelsen. 2026. MikkTSpace. <https://github.com/mmikk/MikkTSpace>
- Yuriy O'Donnell. 2017. FrameGraph: Extensible Rendering Architecture in Frostbite. Game Developers Conference. <https://www.gdcvault.com/play/1024612/FrameGraph-Extensible-RenderingArc>
- Ola Olsson, Markus Billeter, and Ulf Assarsson. 2012. Clustered Deferred and Forward Shading. In *Proceedings of High Performance Graphics*. Eurographics Association, Aire-la-Ville, Switzerland, 87–96. <https://diglib.org/items/6342d4d6-5220-4376-a5c6-a153058f4a3c>
- OpenXR Tutorial contributors. 2026. Graphics. <https://openxr-tutorial.com/linux/vulkan/3-graphics.html>
- Anjul Patney, Marco Salvi, Joohwan Kim, Anton Kaplanyan, Chris Wyman, Nir Benty, David Luebke, and Aaron Lefohn. 2016. Towards Foveated Rendering for Gaze-Tracked Virtual Reality. *ACM Transactions on Graphics* 35, 6, Article 179 (2016), 12 pages. doi:10.1145/2980179.2980246
- Thomas Porter and Tom Duff. 1984. Compositing Digital Images. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '84)*. ACM, New York, NY, USA, 253–259. doi:10.1145/800031.808606
- Ravi Ramamoorthi and Pat Hanrahan. 2001. An Efficient Representation for Irradiance Environment Maps. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '01)*. ACM, New York, NY, USA, 497–500. doi:10.1145/383259.383317
- Ravi Ramamoorthi and Pat Hanrahan. 2002. A Signal-Processing Framework for Reflection. *ACM Transactions on Graphics* 21, 4 (2002), 1004–1042. doi:10.1145/571647.571661
- Nathan Reed. 2021. Visualizing Depth Precision. <https://developer.nvidia.com/blog/?p=32034>
- William T. Reeves, David H. Salesin, and Robert L. Cook. 1987. Rendering Antialiased Shadows with Depth Maps. In *Proceedings of SIGGRAPH 87*. ACM, New York, NY, USA, 283–291. doi:10.1145/37401.37435
- Erik Reinhard, Michael Stark, Peter Shirley, and James Ferwerda. 2002. Photographic Tone Reproduction for Digital Images. *ACM Transactions on Graphics* 21, 3 (2002), 267–276. doi:10.1145/566654.566575
- Resonite Wiki contributors. 2026a. FrooxEngine. <https://wiki.resonite.com/FrooxEngine>
- Resonite Wiki contributors. 2026b. Linux VR Setup. https://wiki.resonite.com/Linux_VR_Setup
- Resonite Wiki contributors. 2026c. The Performance Updates. https://wiki.resonite.com/The_Performance_Updates
- Takafumi Saito and Tokiichiro Takahashi. 1990. Comprehensive Rendering of 3-D Shapes. In *Proceedings of the 17th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '90)*. ACM, New York, NY, USA, 197–206. doi:10.1145/97879.97901
- Christophe Schlick. 1994. An Inexpensive BRDF Model for Physically-Based Rendering. *Computer Graphics Forum* 13, 3 (1994), 233–246. <https://diglib.org/items/581037f7-1e5c-4859-a107-6dfeeb8f3ccb>
- Peter-Pike Sloan. 2008. Stupid Spherical Harmonics Tricks. <https://www.ppsloan.org/publications/StupidSH36.pdf>
- Wojciech Sowa. 2026. Tracy Profiler. <https://github.com/wolfpld/tracy>
- Tomasz Stachowiak and Yasin Uludag. 2015. Stochastic Screen-Space Reflections. SIGGRAPH Courses: Advances in Real-Time Rendering in Games. <https://www.ea.com/frostbite/news/stochastic-screen-space-reflections>
- Unity Technologies. 2015. Camera.Render. <https://docs.unity.cn/520/Documentation/ScriptReference/Camera.Render.html>
- Unity Technologies. 2018. ShaderLab: SubShader. <https://docs.unity.cn/2018.1/Documentation/Manual/SL-SubShader.html>
- Unity Technologies. 2022. Camera.targetTexture. <https://docs.unity.cn/2022.1/Documentation/ScriptReference/Camera-targetTexture.html>
- Unity Technologies. 2026a. Bloom. <https://docs.unity.cn/Packages/com.unity.postprocessing@2.3/manual/Bloom.html>
- Unity Technologies. 2026b. Managing GPU Usage for PC and Console Games. <https://unity.com/en/how-to/gpu-optimization>
- Unity Technologies. 2026c. Render Pipeline Feature Comparison. <https://docs.unity.cn/Manual/render-pipeline-feature-comparison.html>
- Unity Technologies. 2026d. Rendering Order in the Built-in Render Pipeline. <https://docs.unity.cn/Manual/built-in-rendering-order.html>
- Unity Technologies. 2026e. Rendering Paths in the Built-in Render Pipeline. <https://docs.unity.cn/Manual/RenderingPaths.html>
- Unity Technologies. 2026f. RenderTexture. <https://docs.unity3d.com/ScriptReference/RenderTexture.html>

- Unity Technologies. 2026g. ShaderLab: Assigning Tags to a SubShader. <https://docs.unity.cn/Manual/SL-SubShaderTags.html>
- Unity Technologies. 2026h. ShaderLab Command: Blend. <https://docs.unity.cn/Manual/SL-Blend.html>
- Unity Technologies. 2026i. ShaderLab Command: Stencil. <https://docs.unity3d.com/Manual/SL-Stencil.html>
- Unity Technologies. 2026j. ShaderLab: Culling and Depth Testing. <https://docs.unity3d.com/Manual/SL-CullAndDepth.html>
- Unity Technologies. 2026k. Using the Built-in Render Pipeline. <https://docs.unity.cn/Manual/built-in-render-pipeline.html>
- Paul Upchurch and Mathieu Desbrun. 2012. Tightening the Precision of Perspective Rendering. *Journal of Graphics Tools* 16, 1 (2012), 40–56. doi:10.1080/2151237X.2012.649987
- Bruce Walter, Stephen R. Marschner, Hongsong Li, and Kenneth E. Torrance. 2007. Microfacet Models for Refraction through Rough Surfaces. In *Proceedings of the 18th Eurographics Symposium on Rendering*. The Eurographics Association, Goslar, DEU, 195–206. doi:10.2312/EGWR/EGSR07/195-206
- wgpu contributors. 2026. wgpu: Cross-Platform, Safe, Pure-Rust Graphics API. <https://docs.rs/wgpu/>
- wgpu-profiler contributors. 2026. wgpu-profiler. <https://docs.rs/wgpu-profiler>
- Lance Williams. 1978. Casting Curved Shadows on Curved Surfaces. In *Proceedings of SIGGRAPH 78*. ACM, New York, NY, USA, 270–274. doi:10.1145/800248.807402
- World Wide Web Consortium. 2026a. WebGPU. <https://www.w3.org/TR/webgpu/>
- World Wide Web Consortium. 2026b. WebGPU Shading Language. <https://www.w3.org/TR/WGSL/>
- Yellow Dog Man Studios. 2023. Resonite on Steam. <https://store.steampowered.com/app/2519830/Resonite/>
- Yellow Dog Man Studios. 2026a. Renderite. <https://github.com/Yellow-Dog-Man/Renderite>
- Yellow Dog Man Studios. 2026b. Renderite.Candidates. <https://github.com/Yellow-Dog-Man/Renderite.Candidates>
- Yellow Dog Man Studios. 2026c. Renderite.Unity.Renderer. <https://github.com/Yellow-Dog-Man/Renderite.Unity.Renderer>
- Yellow Dog Man Studios. 2026d. Resonite. <https://resonite.com/>
- Yellow Dog Man Studios. 2026e. Resonite.UnityShaders. <https://github.com/Yellow-Dog-Man/Resonite.UnityShaders>
- Fan Zhang, Hanqiu Sun, and Oskari Nyman. 2007. Parallel-Split Shadow Maps on Programmable GPUs. In *GPU Gems 3*. Addison-Wesley, Upper Saddle River, NJ, USA. <https://developer.nvidia.com/gpugems/gpugems3/part-ii-light-and-shadows/chapter-10-parallel-split-shadow-maps-programmable-gpus>